Signals in Linux

A **signal** is an asynchronous notification sent to a process to notify it of an event. Signals are used to handle exceptions, inter-process communication, and process control.

Types of Signals

Signals can be classified into:

1. **Terminal-generated signals** (e.g., `SIGINT` from `Ctrl+C`)
2. **Hardware-generated signals** (e.g., `SIGSEGV` for segmentation faults)
3. **Software-generated signals** (e.g., `SIGUSR1`, `SIGUSR2`)
4. **System signals** (e.g., `SIGKILL`, `SIGSTOP`)

Common Signals:

Signal	Number	Description
`SIGINT`	2	Interrupt from keyboard (`Ctrl+C`)
`SIGKILL`	9	Kill signal (cannot be caught)
`SIGTERM`	15	Termination request (default for `kill` command)
`SIGSEGV`	11	Segmentation fault
`SIGSTOP`	19	Stop process execution (cannot be caught)
`SIGCONT`	18	Continue process execution
`SIGUSR1`	10	User-defined signal
`SIGUSR2`	12	User-defined signal

Sending Signals in C

The `kill(pid, signal)` function is used to send a signal to a process.

Example: Sending a Signal

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

int main() {
    pid_t pid;
    printf("Enter the PID of the process to send SIGUSR1: ");
    scanf("%d", &pid);
```

```
    if (kill(pid, SIGUSR1) == 0) {
        printf("SIGUSR1 sent to process %d\n", pid);
    } else {
        perror("kill failed");
    }
    return 0;
}
```

Run this program and enter a PID of a running process to send `SIGUSR1`.

Receiving Signals

A process can handle signals using the `signal()` or `sigaction()` functions.

Example: Catching a Signal (`SIGUSR1`)

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

// Custom signal handler function
void handle_signal(int sig) {
    printf("Received signal: %d\n", sig);
}

int main() {
    // Register signal handler
    signal(SIGUSR1, handle_signal);

    printf("Waiting for SIGUSR1... (PID: %d)\n", getpid());

    // Infinite loop to keep the process running
    while (1) {
        sleep(1);
    }

    return 0;
}
```

Steps to Run:

1. Compile and run the program:`bash`

```
gcc signal_handler.c -o signal_handler
```

2. `./signal_handler`

3. Open another terminal and find the process ID:

```
ps aux | grep signal_handler
```

4.

5. Send a signal:`bash`

```
kill -SIGUSR1 <PID>
```

Default vs Custom Signal Handlers

- **Default Signal Handlers:** Each signal has a default action (e.g., `SIGTERM` terminates the process).
- **Custom Signal Handlers:** You can override default behavior using `signal()` or `sigaction()`.

Example: Default vs Custom Signal Handling

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

// Custom SIGINT handler
void sigint_handler(int sig) {
    printf("\nCaught SIGINT (Ctrl+C), but not exiting!\n");
}

int main() {
    // Override default SIGINT action
    signal(SIGINT, sigint_handler);
```

```
    printf("Press Ctrl+C to test SIGINT handling (PID: %d)
\n", getpid());

    while (1) {
        sleep(1);
    }

    return 0;
}
```

- Default behavior: `SIGINT` (Ctrl+C) terminates the program.
- With a custom handler: The program prints a message instead.

Ignoring Signals

A process can ignore certain signals using `SIG_IGN`.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int main() {
    printf("Ignoring SIGINT (Ctrl+C) (PID: %d)\n", getpid());

    // Ignore SIGINT
    signal(SIGINT, SIG_IGN);

    while (1) {
        sleep(1);
    }

    return 0;
}
```

- Run this program and press `Ctrl+C`. The process will ignore it.

Summary

Concept	Method
Sending a signal	`kill(pid, signal)`
Handling a signal	`signal(signal, handler)`, `sigaction()`
Ignoring a signal	`signal(signal, SIG_IGN)`
Default behavior	Process termination, stopping, or ignoring

1. `pause()` System Call

The `pause()` system call makes a process **sleep until it receives a signal**.

Syntax

```
#include <unistd.h>

int pause(void);
```

- It **suspends execution** until a signal is received.
- If the signal is caught by a handler, `pause()` **returns -1** and sets `errno` to `EINTR`.
- If the signal terminates the process, `pause()` never returns.

Example: Using `pause()` to Wait for Signals

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

// Signal handler
void handle_signal(int sig) {
    printf("Received signal: %d\n", sig);
}

int main() {
    // Register signal handler for SIGINT
    signal(SIGINT, handle_signal);

    printf("Waiting for signals... (PID: %d)\n", getpid());

    // Pause the process until a signal is received
    pause();

    printf("Exiting after signal reception.\n");
    return 0;
}
```

How to Test

1. Run the program.
2. Open another terminal and send `SIGINT` (Ctrl+C or `kill -SIGINT <PID>`).
3. The process resumes execution after handling the signal.

2. `raise()` System Call

The `raise()` function **sends a signal to the calling process**.

Syntax

```
#include <signal.h>

int raise(int sig);
```

- Equivalent to `kill(getpid(), sig)`.
- Returns `0` on success, or a nonzero value on failure.

Example: Using `raise()` to Send a Signal to Itself

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

// Signal handler
void handle_signal(int sig) {
    printf("Received signal: %d\n", sig);
}

int main() {
    // Register signal handler for SIGUSR1
    signal(SIGUSR1, handle_signal);

    printf("Raising SIGUSR1...\n");

    // Send SIGUSR1 to itself
    raise(SIGUSR1);

    printf("Signal handled successfully.\n");

    return 0;
}
```

How It Works

- The process sends `SIGUSR1` to itself using `raise()`.
- The handler catches it and prints the message.

3. `alarm()` System Call

The `alarm()` system call **schedules a SIGALRM signal** after a specified time.

Syntax

```
#include <unistd.h>

unsigned int alarm(unsigned int seconds);
```

- After `seconds`, `SIGALRM` is sent to the process.
- If `alarm(0)` is called, it cancels any previously set alarm.
- Returns **remaining time** for any previously scheduled alarm.

Example: Using `alarm()` to Trigger SIGALRM

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

// Signal handler for SIGALRM
void alarm_handler(int sig) {
    printf("Alarm signal received! Time's up.\n");
}

int main() {
    // Register signal handler for SIGALRM
    signal(SIGALRM, alarm_handler);

    printf("Setting an alarm for 5 seconds...\n");

    // Set an alarm for 5 seconds
    alarm(5);

    // Pause process until a signal is received
    pause();

    printf("Exiting program.\n");
    return 0;
}
```

How to Test

- The program sets an alarm for 5 seconds.
- After 5 seconds, `SIGALRM` is triggered, invoking the handler.

How to Ignore Signals and Restore Default Handlers in Linux (C Programming)

In Linux, you can:

1. **Ignore a signal** using `SIG_IGN`.
2. **Restore the default behavior** using `SIG_DFL`.

1. Ignoring Signals (`SIG_IGN`)

You can ignore signals by setting their handler to `SIG_IGN`.

Example: Ignore `SIGINT` (Ctrl+C) and `SIGTSTP` (Ctrl+Z)

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int main() {
    // Ignore SIGINT (Ctrl+C)
    signal(SIGINT, SIG_IGN);

    // Ignore SIGTSTP (Ctrl+Z)
    signal(SIGTSTP, SIG_IGN);

    printf("Try pressing Ctrl+C or Ctrl+Z (PID: %d). They
will be ignored.\n", getpid());

    while (1) {
        sleep(1);
    }

    return 0;
}
```

How It Works

- Pressing `Ctrl+C` (`SIGINT`) does nothing.
- Pressing `Ctrl+Z` (`SIGTSTP`) does nothing.

To stop the program, use:

```
kill -9 <PID>
```

or close the terminal.

2. Restoring Default Signal Behavior (`SIG_DFL`)

You can restore a signal's default action using `SIG_DFL`.

Example: Ignore `SIGINT` First, Then Restore It

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int main() {
    // Ignore SIGINT
    signal(SIGINT, SIG_IGN);
    printf("SIGINT is ignored. Try pressing Ctrl+C (PID: %d)
\n", getpid());

    sleep(5);  // Wait for 5 seconds

    // Restore SIGINT default behavior
    signal(SIGINT, SIG_DFL);
    printf("SIGINT default action restored. Now Ctrl+C will
terminate the process.\n");

    while (1) {
        sleep(1);
    }

    return 0;
}
```

How It Works

1. For the first 5 seconds, `Ctrl+C` is ignored.
2. After 5 seconds, default behavior is restored.
3. Pressing `Ctrl+C` now **terminates** the process.